

# BOOK A DOCTOR SYSTEM

Backend Development and Explanation - Project-specific code and flow

This document replaces the generic SkillWallet backend example with the backend used in the Book a Doctor MERN application. It follows Express routes, controllers, services, Mongoose models, JWT middleware, and MongoDB integration.

## 1. server.js - Backend Entry Point

```
require("dotenv").config();

const app = require("./src/app");
const connectDB = require("./src/config/db");

const PORT = process.env.PORT || 5000;

const startServer = async () => {
  try {
    await connectDB();

    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  } catch (error) {
    console.error("Server startup failed:", error.message);
    process.exit(1);
  }
};

startServer();
```

- Loads environment variables from the .env file.
- Connects to MongoDB before starting the Express server.
- Uses port 5000 when PORT is not configured.

## 2. src/app.js - Express Application Configuration

```

const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");

const authRoutes = require("./routes/authRoutes");
const doctorRoutes = require("./routes/doctorRoutes");
const appointmentRoutes = require("./routes/appointmentRoutes");
const adminRoutes = require("./routes/adminRoutes");
const profileRoutes = require("./routes/profileRoutes");

const app = express();

app.use(cors());
app.use(helmet());
app.use(morgan("dev"));
app.use(express.json());

app.get("/", (req, res) => {
  res.status(200).json({
    success: true,
    message: "Book Doctor API is running",
  });
});

app.use("/api/auth", authRoutes);
app.use("/api/doctors", doctorRoutes);
app.use("/api/appointments", appointmentRoutes);
app.use("/api/admin", adminRoutes);
app.use("/api", profileRoutes);

app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: "Route not found",
  });
});

module.exports = app;

```

- Configures CORS, Helmet, Morgan, and JSON request parsing.
- Mounts authentication, doctor, appointment, admin, and profile routes.
- Returns a standard response for unknown routes.

### 3. src/config/db.js - MongoDB Connection

```

const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const connection = await mongoose.connect(
      process.env.MONGO_URI
    );

    console.log(
      `MongoDB Connected: ${connection.connection.host}`
    );
  } catch (error) {
    console.error(
      `MongoDB connection failed: ${error.message}`
    );

    process.exit(1);
  }
};

module.exports = connectDB;

```

- Uses Mongoose and the MONGO\_URI environment variable.
- Stops the process when MongoDB cannot be connected.

## 4. authMiddleware.js - JWT Authentication

```
const jwt = require("jsonwebtoken");

const authMiddleware = async (req, res, next) => {
  try {
    const authorization = req.headers.authorization;

    if (
      !authorization ||
      !authorization.startsWith("Bearer ")
    ) {
      return res.status(401).json({
        success: false,
        message: "Authentication token is required",
      });
    }

    const token = authorization.split(" ")[1];

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    req.user = {
      id: decoded.id,
      role: decoded.role,
    };

    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: "Invalid or expired token",
    });
  }
};

module.exports = authMiddleware;
```

- Extracts and verifies the Bearer token.
- Adds the authenticated id and role to req.user.
- Rejects missing, invalid, and expired tokens.

## 5. adminMiddleware.js - Admin Authorization

```
const adminMiddleware = (req, res, next) => {
  if (!req.user || req.user.role !== "admin") {
    return res.status(403).json({
      success: false,
      message: "Admin access required",
    });
  }

  next();
};

module.exports = adminMiddleware;
```

- Protects admin-only routes after JWT verification.

## 6. User.js - User and Doctor Model

```

const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    firstName: { type: String, required: true },
    lastName: { type: String, required: true },

    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
    },

    password: {
      type: String,
      required: true,
    },

    phone: { type: String, default: "" },

    role: {
      type: String,
      enum: ["admin", "doctor", "patient"],
      default: "patient",
    },

    specialization: String,
    qualification: String,
    experience: Number,
    consultationFee: Number,
    hospital: String,
    address: String,
    about: String,

    available: {
      type: Boolean,
      default: true,
    },

    profileImage: {
      type: String,
      default: "",
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);

```

- Uses one Users collection for Admin, Doctor, and Patient accounts.
- Stores doctor-specific professional fields on doctor-role users.

## 7. Appointment.js - Appointment Model

```
const mongoose = require("mongoose");

const appointmentSchema = new mongoose.Schema(
  {
    patient: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    doctor: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    appointmentDate: {
      type: Date,
      required: true,
    },

    appointmentTime: {
      type: String,
      required: true,
    },

    reason: {
      type: String,
      default: "",
    },

    status: {
      type: String,
      enum: [
        "Pending",
        "Confirmed",
        "Cancelled",
        "Completed",
      ],
      default: "Pending",
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model(
  "Appointment",
  appointmentSchema
);
```

- References one patient and one doctor.
- Restricts appointment status to the four supported values.

## 8. authService.js - Authentication Business Logic

```

const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const registerUser = async (userData) => {
  const {
    firstName,
    lastName,
    email,
    password,
    phone,
    role,
  } = userData;

  const existingUser = await User.findOne({ email });

  if (existingUser) {
    throw new Error("User already exists");
  }

  const hashedPassword = await bcrypt.hash(
    password,
    10
  );

  return await User.create({
    firstName,
    lastName,
    email,
    password: hashedPassword,
    phone,
    role,
  });
};

const loginUser = async ({ email, password }) => {
  const user = await User.findOne({ email });

  if (!user) {
    throw new Error("Invalid email or password");
  }

  const match = await bcrypt.compare(
    password,
    user.password
  );

  if (!match) {
    throw new Error("Invalid email or password");
  }

  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRE || "7d" }
  );

  return {
    token,
    user: {
      id: user._id,
      firstName: user.firstName,
      lastName: user.lastName,
      email: user.email,
      phone: user.phone,
      role: user.role,
    },
  };
};

module.exports = {
  registerUser,
  loginUser,
};

```

- Prevents duplicate user registration.
- Hashes passwords with bcryptjs.
- Creates a role-aware JWT token during login.

## 9. authController.js and authRoutes.js

```
// authController.js
const {
  registerUser,
  loginUser,
} = require("../services/authService");

const register = async (req, res) => {
  try {
    const user = await registerUser(req.body);

    res.status(201).json({
      success: true,
      message: "User Registered Successfully",
      data: user,
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      message: error.message,
    });
  }
};

const login = async (req, res) => {
  try {
    const data = await loginUser(req.body);

    res.status(200).json({
      success: true,
      message: "Login Successful",
      data,
    });
  } catch (error) {
    res.status(401).json({
      success: false,
      message: error.message,
    });
  }
};

// authRoutes.js
router.post("/register", register);
router.post("/login", login);
router.get("/me", authMiddleware, myProfile);
router.put("/profile", authMiddleware, editProfile);
```

- Controllers return structured JSON responses.
- Public login/register and protected profile endpoints are separated.

## 10. appointmentService.js - Appointment Business Logic

```
const Appointment = require("../models/Appointment");
const User = require("../models/User");

const bookAppointment = async (
  userId,
  appointmentData
) => {
  const {
    doctorId,
    appointmentDate,
    appointmentTime,
    reason,
  } = appointmentData;

  const patient = await User.findById(userId);
  const doctor = await User.findById(doctorId);

  if (!patient) {
    throw new Error("Patient not found");
  }

  if (!doctor || doctor.role !== "doctor") {
    throw new Error("Doctor not found");
  }

  return await Appointment.create({
    patient: userId,
    doctor: doctorId,
    appointmentDate,
    appointmentTime,
    reason,
  });
};

const getMyAppointments = async (userId) => {
  return await Appointment.find({ patient: userId })
    .populate(
      "doctor",
      "firstName lastName specialization hospital consultationFee"
    )
    .sort({ appointmentDate: 1 });
};

const getDoctorAppointments = async (doctorId) => {
  return await Appointment.find({ doctor: doctorId })
    .populate(
      "patient",
      "firstName lastName phone email profileImage"
    )
    .sort({ appointmentDate: 1 });
};
```

- Validates both patient and doctor before booking.
- Populates doctor details for patients and patient details for doctors.

## 11. Appointment Status Update



```

const updateAppointmentStatus = async (
  doctorId,
  appointmentId,
  status
) => {
  const appointment =
    await Appointment.findById(appointmentId);

  if (!appointment) {
    throw new Error("Appointment not found");
  }

  if (
    appointment.doctor.toString() !== doctorId
  ) {
    throw new Error("You are not authorized");
  }

  appointment.status = status;
  await appointment.save();

  return appointment;
};

```

- Only the doctor assigned to the appointment can update it through doctor APIs.
- The admin module uses its own protected status-update service.

## 12. appointmentRoutes.js

```

router.post(
  "/",
  authMiddleware,
  createAppointment
);

router.get(
  "/my",
  authMiddleware,
  myAppointments
);

router.get(
  "/doctor",
  authMiddleware,
  doctorAppointments
);

router.put(
  "/:id/status",
  authMiddleware,
  changeAppointmentStatus
);

```

- Supports patient booking, patient history, doctor appointments, and doctor status updates.

## 13. adminRoutes.js - Protected Admin APIs

```
router.get(
  "/users",
  authMiddleware,
  adminMiddleware,
  getUsers
);

router.get(
  "/appointments",
  authMiddleware,
  adminMiddleware,
  getAppointments
);

router.put(
  "/appointments/:id/status",
  authMiddleware,
  adminMiddleware,
  updateAppointmentStatus
);

router.get(
  "/dashboard",
  authMiddleware,
  adminMiddleware,
  dashboardStats
);
```

- Every endpoint requires both authentication and the admin role.
- Provides user, appointment, status-management, and dashboard APIs.

## 14. adminService.js and Dashboard Statistics

```
const User = require("../models/User");

const getAllUsers = async () => {
  return await User.find().select("-password");
};

const getDashboardStats = async () => {
  const totalUsers = await User.countDocuments();
  const doctors = await User.countDocuments({
    role: "doctor",
  });
  const patients = await User.countDocuments({
    role: "patient",
  });

  const appointments =
    await Appointment.countDocuments();

  const pending = await Appointment.countDocuments({
    status: "Pending",
  });

  const confirmed =
    await Appointment.countDocuments({
      status: "Confirmed",
    });

  return {
    totalUsers,
    doctors,
    patients,
    appointments,
    pending,
    confirmed,
  };
};
```

- Excludes password hashes when listing users.
- Calculates live dashboard statistics from MongoDB.

## 15. Complete Backend Request Flow

| Stage       | Book a Doctor implementation                                          |
|-------------|-----------------------------------------------------------------------|
| Client      | React pages call the API through Axios.                               |
| Routes      | Express route files define endpoints.                                 |
| Middleware  | JWT authentication and admin authorization protect routes.            |
| Controllers | Controllers read requests and send JSON responses.                    |
| Services    | Services contain validation, business logic, and database operations. |
| Models      | User and Appointment schemas define MongoDB documents.                |
| Database    | MongoDB stores users, doctor profiles, and appointments.              |

**React Frontend -> Axios -> Express Route -> JWT/Admin Middleware -> Controller -> Service -> Mongoose Model -> MongoDB -> JSON Response -> React UI**

## Conclusion

The backend supports role-based authentication, doctor management, patient management, appointment booking, appointment status updates, profile management, and administrator statistics. Its routes-controller-service-model separation makes the project easier to maintain and explain during the SkillWallet review.